Activity 6: Enforce Validation and Antiforgery Controls

Course: Build ASP.NET Web Apps with Vibe Coding (TGS-2023040477)

Alignment: LO3 · K3/A3

Duration: 50 minutes

Runnable snapshot: src/LocalCRM

Goal

Reject invalid or forged commands while returning accessible recovery guidance.

What you will produce

bounded DataAnnotations, field/summary errors, antiforgery proof and negative-path tests

Safety and data boundary

Use only the supplied fictional records in mock-data/. Do not paste personal, customer, credential, or production data into an AI tool. Review every generated diff before running it.

Vibe script

> Harden the Create Razor Page. Name is required (2–80 characters), Email is required and valid, and Stage is Lead, Qualified or Customer. Add accessible field errors and validation summary. Preserve automatic antiforgery and add tests for invalid input and missing token.

Step-by-step

Step 1 — Add validated input model

Use DataAnnotations on a mutable page input type.

Create or update LocalCRM/Models/CreateContactInput.cs:

```
using System.ComponentModel.DataAnnotations;
namespace LocalCRM.Models;
public sealed class CreateContactInput
{
 [Required, StringLength(80, MinimumLength = 2)] public string Name { get; set; } =
string.Empty;
 [Required, EmailAddress] public string Email { get; set; } = string.Empty;
 [Required] public string Stage { get; set; } = "Lead";
}
```

Step 2 — Guard the handler

Return Page() before calling the store when ModelState is invalid.

Create or update OnPost excerpt:

```
if (!ModelState.IsValid) return Page();
await store.AddAsync(new(Input.Name, Input.Email, Input.Stage), HttpContext.RequestAborted);
return RedirectToPage("./Index");
```

Step 3 — Render errors accessibly

Add asp-validation-summary and asp-validation-for. Ensure visible text tells the learner how to repair the field.

Step 4 — Build and submit invalid values

Try an empty name, a one-character name and a malformed email address.

Expected evidence: Each invalid request redisplays the form and does not call the store.

Step 5 — Inspect the antiforgery token

Open page source and locate __RequestVerificationToken plus the matching cookie.

Step 6 — Prove token enforcement

Send a raw POST without a token.

```
curl -i -X POST http://localhost:5050/Contacts/Create -d
'Input.Name=Aisha&Input.Email=aisha@example.test&Input.Stage=Lead'
```

Expected evidence: HTTP 400; the handler does not persist a contact.

Step 7 — Prove the normal path

Submit via the browser with a valid token and confirm redirect plus stored item.

Step 8 — Run accessibility regression

Run axe and keyboard checks on both initial and error states.

Run the supplied snapshot

The Activity folder already contains a complete stage-appropriate ASP.NET Core project and mock data. From this Activity folder:

```
cd src/LocalCRM
dotnet restore
dotnet run --urls http://localhost:5106
```

Open http://localhost:5106. For Activities 7–10, also probe http://localhost:5106/api/contacts; for Activities 8–10, probe /openapi/v1.json and /health/ready.

Acceptance checklist

□ The project targets net10.0 and restores without an unreviewed package.

□ The app starts from this Activity folder without relying on another Activity.

□ The supplied mock contacts are fictional and load as described.

□ Positive and negative paths match the expected status or UI state.

□ The diff contains only intended files and no secret.

Troubleshooting

- NETSDK1045: install a .NET 10 SDK or select the installed 10.0 feature band in global.json.
- Address already in use: stop the previous Activity process or use --urls http://localhost:PORT with a free port.
- HTTP 404: compare the exact route, method, route constraint and Activity stage.
- HTTP 400/415: inspect ProblemDetails and confirm JSON shape plus Content-Type: application/json.
- SQLite failure (Activities 8–10): remove only the local training localcrm.db, then restart so the snapshot recreates mock data.

Reflection and evidence record

1. Explain how the evidence demonstrates LO3 and K3/A3.

2. Name one AI suggestion you rejected or changed and why.

3. Identify one negative path you tested and the observable result.

4. State the next improvement you would make without widening the approved scope.

Evidence item	Record the path, command result or observation
Build or test result	
Runtime URL and HTTP status	
Positive-path evidence	
Negative-path evidence	
Reviewed AI change or rejection	
Commit or evidence reference	